If the method you're stubbing takes arguments (such as the cost_factor), RSpec passes these as additional parameters into your block.

Since both and_call_original and and_wrap_original need an existing implementation to call, they only make sense for partial doubles.

When You Need More Flexibility

So far, we've seen several different ways to customize how your test doubles behave. You can return or yield a specific sequence of values, raise an exception, and so on.

Sometimes, though, the behavior you need is slightly outside what these techniques provide. If you're not quite sure how to configure a double to do what you need, you can supply a block containing whatever custom behavior you need. Simply pass the block to the last method call in the receive expression.

For instance, you might want to simulate an intermittent network failure while you're testing. Here's an example of a weather API test double that succeeds 75 percent of the time:

14-customizing-test-doubles/01/configuring_responses.rb

```
counter = 0

allow(weather_api).to receive(:temperature) do |zip_code|
  counter = (counter + 1) % 4
  counter.zero? ? raise(Timeout::Error) : 35.0
end
```

When your code calls weather_api.temperature(some_zip_code), RSpec will run this block and, depending on how many calls you've made, either return a value or raise a timeout exception.

Don't Get Carried Away with Blocks



If your block gets any more complex than the weather API example here, you might be better off moving it into its own Ruby class. Martin Fowler refers to this kind of stand-in as a *fake*.[3] Fakes are particularly useful when you need to preserve state across multiple method calls.

Setting Constraints

Most of the test doubles you've created will accept any input. If you stub a method named jump with no other options, RSpec will use your stub whenever your code calls jump, jump(:with, :arguments), or jump { with_a_block }.

3. https://www.martinfowler.com/bliki/TestDouble.html